

Contents

The Driving Floor: When an External Symbolic Reasoning Board Helps an LLM	1
Exact Reasoning from Cheap Quantized Local Models	1
Abstract	2
1. Introduction	2
2. Related Work	3
3. The rulith Board	4
3.1 Working memory and stratified closure	4
3.2 Truth maintenance	4
3.3 Exact-or-fail arithmetic	4
3.4 Actions	5
3.5 The two gates (propose / adjudicate)	5
3.6 The grounding floor	5
3.7 Functional-dependency conflicts	5
4. The Driving Floor	6
4.1 Driving is not free-reasoning	6
4.2 The lift relation	6
4.3 The thinking $\times$ driving 2×2	7
4.4 The empty-board failure mode	7
4.5 The driving floor is a training property — not bit-width, not active compute	7
5. Experimental Setup	8
5.1 Models	8
5.2 Task families	8
5.3 Arms	10
5.4 Metrics	10
5.5 Honesty of the setup	10
6. Results	10
6.1 Per-family behaviour	11
6.2 Lift tracks the error rate	12
6.3 The driving floor (a core result, not a limitation)	12
6.4 The sweet spot — and the lowest-compute anchor	12
6.5 Quantization (resolved)	12
6.6 Cost — and the redundancy of thinking	12
6.7 Belief revision	13
7. Limitations & Threats to Validity	13
8. Conclusion	14
Appendix: full results matrix	15
Acknowledgments	15
References	15
Appendix — Reproducibility	16

The Driving Floor: When an External Symbolic Reasoning Board Helps an LLM

Exact Reasoning from Cheap Quantized Local Models

Victor Shaw · Independent Researcher · michaltina@hotmail.com

Pilot study / technical report — release draft. The contribution is a framework + an open system + pilot evidence. The cross-model and quantization sweep is converged; a fuller campaign — tool-augmented baselines, multi-seed statistics, broader tasks, and component ablations — remains future work (§7). As of 2026-06-21 the converged, trusted datapoints span cloud `deepseek-v4-flash`, local `qwen3.6-27b` (4-bit/fp16/fp8), `gemma-4-31b-qat` (official 4-bit), `gpt-oss-120b` (official 4-bit), and the

3B-active qwen3.6-35b-a3b anchor; the fp16/fp8 quantization gradient is now in (a null result, \$6.5). gemma-4-12b / qwen3.5-9b are reported as boundary probes (\$5.5). n=50/family, single machine. — Victor Shaw, 2026-06.

Abstract

Large language models remain unreliable at *structural* reasoning — exact arithmetic, keeping a claim consistent with its evidence, not reporting a conclusion they never derived — even as they grow stronger at open-ended judgment. We study **rulith**, a *propose-adjudicate* symbolic *reasoning board*: the model proposes facts, rules, and actions; the board runs stratified-negation Datalog closure, truth maintenance, and exact (*exact-or-fail*) arithmetic, and **gates** any conclusion it did not derive.

Two facts frame our results. **First, the board’s guarantee is structural and model-independent.** Its arithmetic is exact, and its certifications are earned by derivation rather than by model assertion, across cloud and quantized-local deployments, four structural task families, and $n = 50/\text{family}$. On the trust probe, **no model could make the board certify an unfixed result** (false-certify = 0% on coding-trust, *even for models whose driving collapses*); the board fails *closed*, never *open*. The board’s value is therefore a trust property, not merely a score.

Second — our main finding — whether a model can obtain that trust property is set by a measurable capability we call the driving floor. This is the model’s ability to *drive* the symbolic interface. The driving floor is **separable from free reasoning** and tracks the model’s training generation (and total capability) — not its bit-width, not its per-token active compute, and not whether it “thinks.”

Concretely, the pilot supports a simple organizing relation: **board lift $\approx$ free-reasoning error rate $\times$ driving success.** A model that reasons poorly but drives cleanly can *recover* exact reasoning — a 4-10% $\rightarrow$ 100% hard-arithmetic lift on cheap local models, cleanest at **3B active params**. A model that cannot drive gets nothing, and on tasks that demand an *earned* result can do worse than bare chat. We therefore report an honest map: where the board lifts, where it merely adds a trust guarantee, where it backfires, and how to choose a board-matched model. A structural-class corollary is that **thinking is redundant on the board**: board + thinking ties board + non-thinking at 100, while a *bare* thinking model spends 5-22 $\times$ the tokens and still misses the board’s ceiling. We open-source the system and reproducible probes. This is a pilot; the full campaign is future work, and we do **not** claim to have invented the propose-verify loop.

1. Introduction

LLM failures split in two. *Capacity* failures — semantic judgement, imagination, world knowledge — live in the weights, and no external tool fixes them. *Structural* failures — a wrong product inside a multi-line sum, a claim inconsistent with the evidence on the table, an “I fixed it” that was never verified — are mechanical, and they are exactly the class a faithful deductive engine can eliminate. This paper is about the second kind, and about a system that removes it not by making the model smarter but by moving the mechanical half of reasoning onto an external board that executes it exactly and refuses to be fooled.

It is by now close to folk wisdom in agentic systems that *the harness often matters more than the model*: holding the model fixed, harness and scaffolding choices move task scores substantially. That makes a question conspicuous by its absence: **when does a symbolic harness actually pay off, and for whom?** A strong model may need little help; a model that cannot operate the harness gets none. This paper maps that boundary for one concrete symbolic board.

What does such a board actually buy you? Not “better answers on average” — a **trust guarantee**. The board’s arithmetic is exact, and its conclusions are gated: it will not report a result it did not derive, and it will not certify a fix that was never made. Crucially, this guarantee **does not depend on the model**. Across every deployment we test — cloud, full-precision, and cheap quantized local — *no model can make the board certify an unfixed result* (false-certify = 0%), and when a

weak model’s driving collapses the board returns *nothing* rather than something false: it fails **closed**, never **open**. An exact, derivation-backed result that holds regardless of which cheap model you point at it is the prize.

There is a catch, and it is the subject of this paper. The board only reasons over what the model **feeds** it: to claim the guarantee, the model must *drive* the symbolic interface — turn a problem into facts and rules, emit well-formed operations, and recover when the board rejects one. This *driving* ability is **separable from free reasoning**: a model can be poor at mental arithmetic yet drive flawlessly (and so inherit the board’s exact arithmetic), or strong at reasoning yet unable to drive at all. The surprising part is *what* sets it: in our pilot the driving floor tracks a model’s **training generation**, not its bit-width and not whether it “thinks.” A cheap 4-bit local model that drives cleanly turns 4%-correct hard arithmetic into 100%; a higher-bit model that cannot drive gets nothing. We summarize this as a predictable relation — **board lift $\approx$ free-reasoning error rate $\times$ driving success** — and devote the paper to characterizing it.

We answer with a propose/adjudicate framing. The model **proposes**: it models the problem as facts and rules and drives the interface across turns. The board **adjudicates**: it derives the closure, maintains consistency, computes arithmetic exactly, and refuses any conclusion that was not derived. The model supplies intuition; the board supplies the deductive half — exactly and unforgeably.

Contributions. 1. **The driving floor — our main contribution.** The binding constraint on an LLM + symbolic-board system is not the model’s reasoning but its ability to *drive* the interface: emit valid operations, model the problem into the board’s vocabulary, and recover from rejection. We show this property is **separable from free reasoning** (a model can reason poorly yet drive cleanly, and the reverse) and tracks training generation, not bit-width or thinking budget. We give an organizing relation — **board lift $\approx$ free-reasoning error rate $\times$ driving success** — and the thinking $\times$ driving 2×2 it implies. 2. **A structural, model-independent trust guarantee — the stakes.** Because the board reasons deductively and gates every conclusion, its results are exact and its certifications derivation-backed *independent of the driving model’s quality*: false-certify = 0% across all models, including those whose driving collapses. Collapsed driving fails *closed* (an empty board, no result), never *open* (a fabricated one). This **decouples trustworthiness from model capability** — and is what makes the driving floor worth caring about: what you drive *toward* is a checkable guarantee. 3. **A quantization / compute result.** Conditional on driving, the board’s output is **invariant to quantization and to think/nothink** — the board, not the model, does the reasoning. And on the *driving* side, neither bit-width nor per-token (active) compute sets the floor: a **3B-active** current-generation model drives as cleanly as a dense 27B, while larger-active older models collapse — the floor is a training property (§4.5). 4. **An open system, an honest map, and a selection rule.** We open-source rulith with reproducible probes; we report where the board lifts, where it merely adds a trust guarantee, and where it backfires (negative results included); and we give a practical rule for choosing a board-matched model: favor a current-generation model that drives cleanly. Instruction-following is the observable proxy; raw reasoning strength matters less, because the board supplies the mechanical reasoning.

2. Related Work

The circuit we use — a model proposing into a symbolic validator that gates unsafe or underived conclusions — has clear precedents. We do **not** claim the propose-verify loop itself. *Constraint-checked / propose-verify execution*: G-SPEC [G-SPEC] constrains LLM agents with a knowledge graph + SHACL gate and shows in ablation that the symbolic validation drives the majority of the safety gain; CEGIS-style LLM+SAT loops [CEGIS] and neuro-symbolic verification for process control follow the same propose-then-check shape. *Compiling to deterministic code*: Compiled AI [CompiledAI], PlanCompiler [PlanCompiler], and Blueprint-First [Blueprint] decouple workflow logic from the generative model and execute a deterministic artifact. *Grounding / provenance*: claim-evidence tracing and execution-provenance work [PaperTrail, AgentTraces] tie conclusions back to sources.

Our novelty is therefore **not** the propose-verify loop. It is (i) the empirical **driving-floor** characterization, (ii) board-lift evidence on **quantized local models**, and (iii) the **grounding floor** (weakest-premise tiering + honesty-as-residual). We meet three likely objections directly:

- “Isn’t this G-SPEC re-skinned?” The loop is not our contribution; the empirical characterization (driving floor / quantization / when-it-helps), a single general-purpose board, and the grounding floor are.
 - “The board can’t verify the number-one failure — whether the spec or the premises are right.” Conceded. The grounding floor makes this explicit, human-anchored, and marks the weakest premise (honesty as residual). We *mitigate*, not *eliminate*, and we say so plainly.
 - “Why a whole Datalog board rather than better prompting or a frontier model?” Because the exact arithmetic and the non-fabricable gate are quantization-independent and recover exact reasoning from cheap quantized models with a derivation-backed certificate — which prompting and frontier models do not provide — and we measure it.
-

3. The rulith Board

rulith is a *propose-adjudicate* symbolic reasoning board. A language model proposes facts, rules, and actions; the board performs forward closure, maintains consistency, computes exact arithmetic, and — crucially — *refuses* any conclusion that was not derived. The division of labour is sharp: the model supplies intuition (what to model, how to drive the interface); the board supplies the mechanical, deductive half. It does not make the model “think”; it makes the model’s structural commitments exact, auditable, and non-fabricable on the large class of decidable problems.

The design follows a three-layer discipline (foundations.md): a faithful deductive core (everything in this section), a heuristic layer that may only *propose* and whose products must return to the core for re-validation, and engineering scaffolding (atomicity, concurrency tokens, context windowing) that is explicitly *not* allowed to masquerade as reasoning. Only the first layer is described below; it is the part that carries the guarantees.

3.1 Working memory and stratified closure

The board state is a predicate working memory. Rules (`add_axiom`) fire over it under forward chaining with stratified negation — Datalog with stratified NAF — materializing a layered closure. All mutation flows through a single entry point that validates, normalizes (numeric strings are canonicalized only on a round-trip-safe basis), and gates derivation, so there is exactly one place where the consistency contract is enforced.

3.2 Truth maintenance

Every derived fact carries `evidenceRefs`: a justification graph in the JTMS/ATMS tradition. When support is withdrawn, dependent derivations are retracted with it. The board distinguishes two kinds of removal, and the distinction is load-bearing: **consumption** is *archival* — the fact “was once true, now used up” — whereas **retraction** is physical deletion with an evidence cascade — “never true”. Each apply records an event; events reference only the action node, so availability follows `evidenceRefs` recursively and an event that cites an archived fact correctly becomes invisible rather than dangling.

3.3 Exact-or-fail arithmetic

Comparison, arithmetic, and string builtins are total functions under an *exact-or-fail* contract: integers within $\pm 2^{53}$ are computed exactly; overflow, Infinity, or NaN **fail the literal** rather than silently returning a rounded value that looks exact. Arithmetic literals are ordered automatically by data dependency, so a model may write a chained computation out of order and the board still derives it. This is the board’s answer to the single most common structural failure of language

models — mental arithmetic — and, unlike the model, the board never produces a quietly-wrong “exact” number.

3.4 Actions

Actions are production-rule right-hand sides (the OPS5 add/remove-WME lineage): a guarded transform that consumes (negative effect) and produces (positive effect) working-memory facts. They are the non-monotone half of the system but remain inside the production tradition; their effects are simulated on a cloned board before commit, so a whole ordered plan can be dry-run and drift-guarded before any state changes.

3.5 The two gates (propose / adjudicate)

The propose/adjudicate contract is enforced by two gates, and they are the hard core of the system:

- **Derivation gate.** A guarded conclusion — e.g. a `finding(kind=fixed)` in a coding task — must be *closure-derived* from its evidence, not asserted by fiat. The board tags every fact by provenance: `[derived]` (closure-earned), `[effect]` (an action product the model constructed), `[asserted]` (a bare model claim). Only `[derived]` certifies; the other two never satisfy a guard.
- **Done gate.** A non-empty board may not “finish” without a recorded, *derived* result. A self-sealed goal — one the model asserted rather than earned — is refused at the finish line.

This is the project’s top-level thesis applied recursively *inside* the kernel: heuristics propose, deduction adjudicates. A model cannot get credit for a conclusion by stating it; it must put the primitive observation on the board, add a rule that derives the conclusion, and let the closure produce it — or the gate refuses to close.

3.6 The grounding floor

The board guarantees that the *inference* from premises to conclusion is faithful and exact; it does **not** guarantee that the premises are true. Ground facts — a weight, a fraud amount, a measured metric — are the system’s interface to the world, and logic cannot reach outside itself to verify them. Evidence therefore splits by *verifiability* into three tiers, a line that cuts across every domain (a single coding task contains all three at once), not along domain boundaries:

1. **derived** — the closure can re-derive it; it does not depend on honesty, because the board checks it directly and a lie is caught on the spot.
2. **attested** — it entered through an unforgeable channel: a machine-attested predicate that only the harness/tool may write, or a fact whose `createdBy` is `tool/system`. Trust rests on a `*named source`
 - `channel*`, not on the model’s general honesty.
3. **asserted** — the model put it on the board itself. Honesty is load-bearing only in this tier; it is the residual the board can expose but not remove.

`groundingOf` walks a conclusion’s `evidenceRefs` down to its ground premises and reports the weakest tier plus the witnesses: a conclusion is only as trustworthy as its weakest ground fact. The mental model is *rules of evidence*, not a trusted agent — a court does not believe the prosecution’s stated figure; it admits records with a chain of custody and lets the judge reason faithfully to the verdict. **Honesty is a residual, not a foundation:** every evidence chain bottoms out at some observation, and the board makes that layer small, sourced, and explicitly marked, but cannot make it vanish.

3.7 Functional-dependency conflicts

A domain may declare a functional dependency, `functional_dependency(predicate, key)`, asserting that within a predicate the key arguments determine the rest (e.g. a given invoice item has exactly

one cost). The kernel only *adjudicates*: if two facts share the declared key but disagree on the value, it raises a conflict, taints both facts as *disputed*, surfaces it in the standing critique, and `derive_aggregate` **refuses** a conflicted source rather than silently double-counting it. Declaring a dependency can only *tighten* the board (it can never launder a conclusion into existence), so — unlike attestation — it needs no trusted-channel gate; a model declaring its own dependencies is safe. This closes a class of *modeling pollution* we observed in practice: a model that bare-asserts a value it should have let the board derive (a hand-computed `cost`) now *collides* with the derived value and is flagged, instead of poisoning a sum.

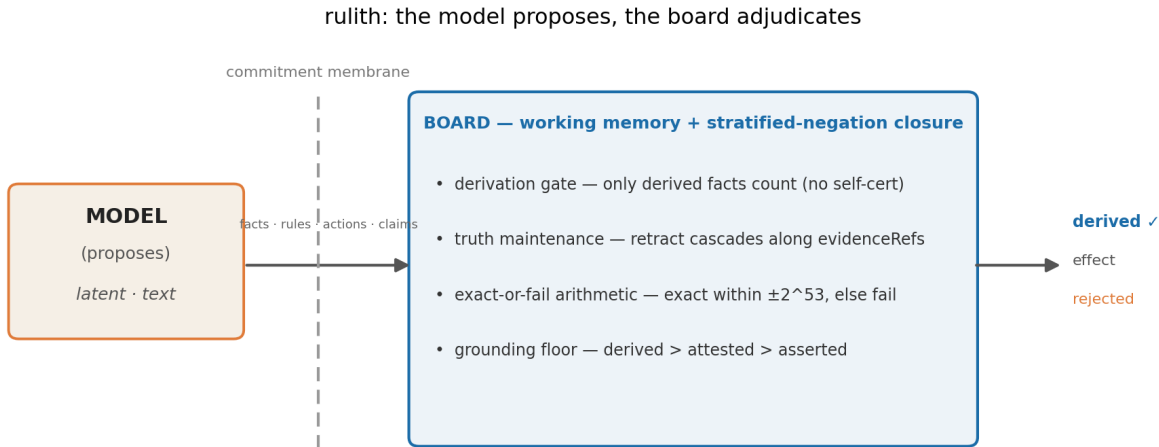


Figure 1: The rulith board. The model *proposes* (latent · text); only commitments that cross the *commitment membrane* into the board are *adjudicated* — stratified-negation closure, the derivation gate, truth maintenance, exact-or-fail arithmetic, and the grounding floor — and emerge as derived / effect / rejected.

4. The Driving Floor

The board’s value is not uniform across models. Whether a model can *extract* that value turns on a capability we name the **driving floor**. It is separable from free-reasoning ability, and — our sharpest pilot finding — it is set by the model’s training (generation + total capability), not by bit-width or per-token compute.

4.1 Driving is not free-reasoning

We separate two model capabilities:

- **Free reasoning** — solving the task in-weights: doing the arithmetic, spotting the bad ledger row, knowing whether a fix is real.
- **Driving** — emitting well-formed board operations, *modeling* the problem as facts and rules, and advancing across turns until the board certifies a result.

The board substitutes for the mechanical/deductive half of free reasoning. It does **not** substitute for driving: if the model cannot feed the board, the board never receives the data and the substitution never happens. Driving is a protocol skill, largely orthogonal to how well the model reasons unaided.

4.2 The lift relation

We summarize the pilot with a qualitative organizing relation:

board lift $\approx$ (free-reasoning error rate) $\times$ (driving success rate).

The first factor says the board can only recover errors the model would otherwise make: on a dimension where the model is already reliable, the board contributes a *structural guarantee* (an unforgeable, derivation-backed result) rather than a higher score. The second factor says the lift is gated on driving: a high error rate buys nothing if the board stays empty. Both factors are necessary; the product form predicts where the board helps, where it merely guarantees, and where it cannot help at all.

4.3 The thinking $\times$ driving 2 $\times$ 2

The two factors define a 2 $\times$ 2 over *free-reasoning error* (low/high) and *driving* (clean/collapsed). The sweet spot is **high error $\times$ clean driving**: a model whose free reasoning is weak — so there is much to recover — but whose driving is clean — so the recovery is realized. Our anchor datapoint sits squarely there: qwen3.6-35b-a3b — a **3B-active** MoE — takes hard multi-line invoice arithmetic from **baseline 6% to 100%** (and audit 24% $\rightarrow$ 100%) under the board, with **zero empty boards** across all three task families (n=50); the dense 4-bit qwen3.6-27b is the same story (4% $\rightarrow$ 100%). The model contributes essentially none of the correct arithmetic; the board contributes all of it, and the model only models and drives. There is no cleaner demonstration of “the board substitutes for reasoning” than a model that cannot do the arithmetic at all becoming exact once connected to the board.

The other quadrants are instructive as *honest* results. A strong/thinking model sits at low-error $\times$ clean-driving: the board draws even on score and adds structural guarantees, because there is little error to recover. Thinking is then *redundant*: board + non-thinking already ties board + thinking, at a fraction of the tokens (\$6.6). A model whose driving collapses sits in the right-hand column regardless of its error rate. The board cannot help, and on tasks where it is asked to certify, a collapsed driver can even be *worse* than bare chat. These “when it does not help / when it hurts” cases are part of the contribution, not footnotes.

4.4 The empty-board failure mode

The characteristic driving failure is the **empty board**: the model’s operation produced no new facts — a malformed op, a wrong predicate or argument, a rule that did not fire — and the model does not notice, then loops blind, often to the turn limit. It is the dominant failure mode for weak drivers and it tracks driving success directly. At the boundary it is usually **format lock-in**: a single malformed-JSON slip (a doubled bracket, a missing brace, prose leaking into the call) fails to parse, and because the transcript is append-only the model *re-reads its own bad output and repeats it* — an autoregressive trap that, untreated, burns to the turn limit (in one observed case, 70 minutes on a single item). Tellingly the reasoning underneath is often intact: at the boundary, arith failures are *all* empty boards with **zero** non-empty arithmetic errors (Table 1) — the model could do the task; it could not get a clean op past the parser.

Two countermeasures matter. (i) A per-turn “ Δ since your last op” line that, on an empty delta, says **NOTHING CHANGED** — turning a silent failure into an actionable teaching signal so the model fixes or retries instead of repeating the dead call. (ii) Honest accounting: the empty-board *metric* counts only a **truly empty** board (zero derived facts), never a full-but-*wrong* one — a wrong answer is scored as a reasoning error, not miscredited as a driving collapse. The residual question — how much of a boundary collapse is the model versus a brittle harness — is a measurement choice we make explicit in §4.5 and §5.5.

4.5 The driving floor is a training property — not bit-width, not active compute

The natural worry is that cheapness — fewer bits, or fewer *active* parameters — buys a collapse in driving. The pilot rejects it on **both** axes. *Bit-width*: within an official quantization, precision is irrelevant to driving — qwen3.6-27b reaches arith/audit 100 at 4-bit, fp16 and fp8 alike, with zero empty boards in every case (the board’s output is exact regardless of model precision). *Active*

compute: the lowest-compute clean driver is a **3B-active** MoE (qwen3.6-35b-a3b: arith 100 / audit 100), while a **12B-active** dense model (gemma-4-12b) and a **9B-active** one (qwen3.5-9b) collapse into format lock-in — *more* per-token compute, *worse* driving. A second MoE sharpens the point within the same architecture class. The **4B-active** gemma-4-26b-a4b has more active parameters than the 3B-active anchor but belongs to an older generation; it drives only *partially* (78 / 70 on arith / audit), not to 100. What separates them is total capability and training generation (a current-gen 35B-total model vs older 9–12B ones), not bit-width or compute-per-token. **The floor is a property of the model’s training, not of how cheaply it is served.**

We flag a correction, because it bears on honesty of measurement. An earlier reading of this pilot — that a *crude quantization* collapses driving — was a harness artifact: a brittle driver loop (uninformative parse errors, no reset) turned a single malformed-JSON slip into an autoregressive lock-in and an empty board (§4.4). Once the harness localizes the parse error and bounds the loop, the apparent quantization/compute effect dissolves and the clean-driving boundary snaps back to the model-training line. The same model that looked like a “crude 4-bit collapse” (qwen3.6-35b-a3b: arith/audit 72/42) drives to 100/100 post-fix — it was always capable; the harness was hiding it. We report the floor as a training property and disclose the harness policy (§5.5).

This is the strongest form of the cheap-deployment story: exact, non-fabricable reasoning is recoverable at **3B-active, 4-bit, local** compute — a current-generation small model drives the board as cleanly as a dense 27B, at a fraction of the per-token cost.

The driving floor is one of a *family of floors* the board marks but cannot cross (foundations.md): the **grounding floor** (a conclusion is no stronger than its weakest premise), the **frontier floor** (self-driven progress is bounded by the completeness of the frontier model, which is itself asserted), and the **driving floor** (liveness is exogenous — the board can *measure* a stall and escalate, but cannot *verify* a model into driving). The board’s honesty is not that it guarantees success; it is that, at each step, what can be proved is proved and what cannot be judged is marked. Marking what it cannot guarantee is itself the contribution.

5. Experimental Setup

5.1 Models

The pilot spans cloud full-precision and local quantized serving, so that quantization is a variable rather than a confound. Each run is labelled with slug, quantization, mode, and endpoint in data-manifest.md to prevent model-identity drift. Local models are served through LM Studio; cloud models through their provider API.

Data maturity (stated honestly): the convergence-stable, trusted datapoints are the **3B-active** qwen3.6-35b-a3b, qwen3.6-27b (4-bit/fp16/fp8), gemma-4-31b-qat (official 4-bit), gpt-oss-120b (official 4-bit), the partial-driver gemma-4-26b-a4b, and the cloud **deepseek-v4-flash** arms (non-thinking -chat and thinking -reasoner; the strongest *audit* evidence). The gemma/qwen **fp16/fp8** gradient is now in and resolves to a **null result** (§6.5). gemma-4-12b and qwen3.5-9b are reported as **boundary probes** on the strict harness (§5.5). We **exclude** one cloud run as unusable: a pre-fix deepseek-flash-v4 whose rule-laundering bug inflated its coding false-positive rate (caught, fixed, regression-tested; §7).

5.2 Task families

Four probes, each isolating one structural failure; together they span **computation, numeric inference, trust, and revision**. **arith** — exact multi-line invoice arithmetic at a hard setting (8-digit unit × 5-digit quantity, multiple lines), targeting mental-arithmetic error. **audit** — ledger error detection scored as false-positive (hallucinated finding) and false-negative (missed error) rates. **coding-trust** — interception of a false “I fixed it” claim: a certified fix must be backed by machine-attested evidence, not prose.

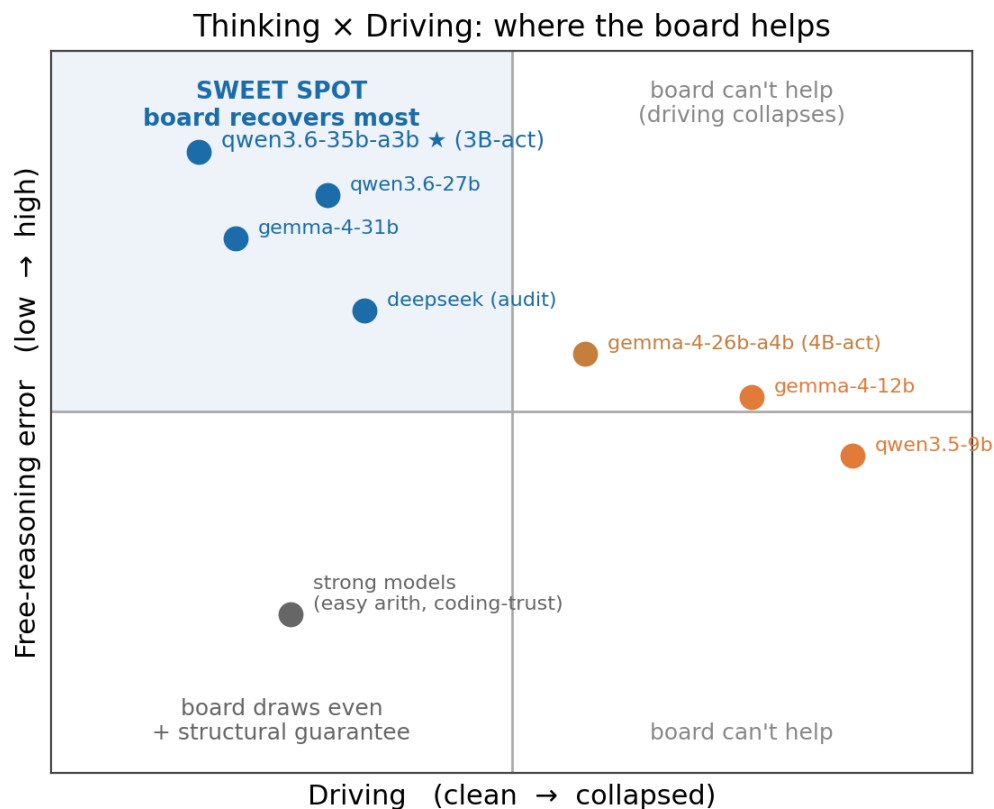


Figure 2: The thinking × driving 2×2. The *sweet spot* — high free-reasoning error × clean driving — is where the board recovers the most; our anchor qwen3.6-35b-a3b (3B-active) sits there. Models whose driving collapses (boundary probes gemma-4-12b, qwen3.5-9b) lie in the right column, where the board cannot help regardless of error.

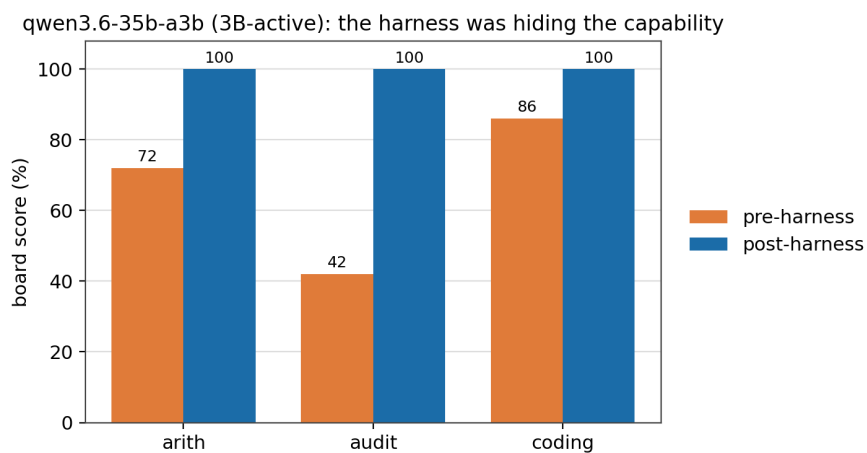


Figure 3: The harness correction (\$4.5) on qwen3.6-35b-a3b (3B-active): the *same* model reads 72 / 42 / 86 under a brittle driver loop, and 100 / 100 / 100 once the harness localizes parse errors and bounds the loop — it was always capable; the harness was hiding it.

revision — belief revision under retraction. An access-control world is presented as ground facts (a user’s roles; which roles grant which permissions; which permissions enable which actions) plus two derivation rules (role $\wedge$ grant $\rightarrow$ permission, permission $\wedge$ enable $\rightarrow$ action); the closure derives which actions the user *can do*. A fact is then **retracted** (a role or grant revoked), and the task is to report which derived conclusions still hold. The discriminating cases are conclusions with **redundant support** — an action enabled through two independent permission paths, one of which is revoked — which must **survive**, alongside single-support conclusions that must be **withdrawn** and independent conclusions that are **unaffected**. This exercises the board’s truth-maintenance layer (a retraction cascades along `evidenceRefs`, and a conclusion is withdrawn iff *no* surviving derivation remains) — a non-numeric capability distinct from the computation (arith, audit) and trust (coding) families. We sweep problem size: 8–27 queried conclusions per case and 1–6 retractions.

5.3 Arms

Two arms per task. **baseline** — bare chat, judged from prose (e.g. ANSWER: N). **board** — the model drives rulith, scored *structurally from board state*: a per-line cost and grand total must appear as [derived] facts (an asserted-only board is a FAIL), a flagged row must be [derived], a certified fix must carry a derived `finding(kind=fixed)`. The board arm cannot score by assertion.

5.4 Metrics

Per-family exact-set solve rate, false-positive prevention, the **empty-board / driving-failure rate** (operations that produced no facts), and token tallies (in/out/calls), with Wilson 95% confidence intervals; we mark a board-vs-baseline difference when the intervals are disjoint.

5.5 Honesty of the setup

n=50/family, single machine, single run per cell — a pilot. The scorer itself was audited and corrected (§7): its baseline negation judgement once misread an honest “not yet fixed” as a false claim, which would have inflated the board’s apparent win; this was caught, fixed, and regression-tested. We treat *auditing our own measurement* as part of the contribution.

Harness policy. Driving is measured through a fixed driver loop, so we state its policy. Clean-tier numbers are *post-harness* — after committed fixes to format-error reporting, the empty-board teaching signal (§4.4), and the empty-board *metric* correction (truly-empty vs full-but-wrong). The boundary probes (gemma-4-12b, qwen3.5-9b) are reported on this **strict** harness with a deliberate omission: we do *not* add per-model format-tolerance or loop-reset, so a malformed-JSON slip that locks a model into an empty board counts as a driving failure even when the underlying reasoning is correct. Their rates are therefore a **disclosed lower bound**; the bucketed evidence (Table 1: boundary failures are *all* empty boards, zero non-empty arithmetic errors) shows the deficit is format, not reasoning, and a fairer harness would lift them. We chose a uniform strict harness plus this disclosure over a per-model harness, which would break cross-model comparability.

6. Results

Table 1 — converged pilot matrix (post-harness). Cell = board% / baseline%; * = Wilson 95% CIs disjoint (board vs baseline). Empty-board = the driving-floor signal (arith/audit), as a count. false-cert = board certifying an unfixed bug (the structural-trust floor). Clean-tier rows n=50/family; **boundary probes** are small-n (8–17), reported on the strict harness (§5.5). Full results matrix in the Appendix. *Precision:* unlabeled local rows are 4-bit, non-thinking; fp16/fp8 and cloud rows carry precision and mode (nt/think); gpt-oss-120b is lowthink; cloud = full-precision.

model	arith b/base	audit b/base	coding cert	empty-bd (ar/au)	false-cert
qwen3.6-35b-a3b (3B-active MoE)	100/6 *	100/24 *	100/100	0 / 0	0%
qwen3.6-27b	100/4 *	100/100	100/100	0 / 0	0%

model	arith b/base	audit b/base	coding cert	empty-bd (ar/au)	false-cert
deepseek-v4-flash (nt)	100/98	100/52 *	100/100	0 / 0	0%
deepseek-v4-flash (think)	100/86	100/26 *	100/100	0 / 0	0%
gemma-4-31b	100/4 *	92/90	100/100	0 / 0	0%
gpt-oss-120b (lowthink)	96/88	86/56 *	100/100	0 / 0	0%
<i>boundary probes (small n):</i>					
qwen3.5-9b (9B dense)	22/6	27/33	100/100	14 / 5	0%
gemma-4-12b (12B dense)	50/0 *	17/42	100/100	6 / 2	0%

Reading the table. **(1) Anchor:** qwen3.6-35b-a3b, a **3B-active** MoE, drives to **arith 100 / audit 100** (baseline 6 / 24) at 0 empty boards — the lowest per-token compute in the table, and the lowest-compute *clean* driver we found. **(2) Empty-board predicts the score:** 0 empty → the board lifts or ties; the probes’ empties are the driving collapse — and they are **format lock-in, not reasoning** (every probe failure is an empty board; *zero* non-empty arithmetic errors), so a fairer harness would recover them. We report them on the strict harness as a disclosed lower bound (\$5.5). **(3) Driving $\neq$ per-token compute:** the 3B-active MoE drives clean while the **12B-active** gemma-4-12b and **9B-active** qwen3.5-9b collapse — *more* active compute, *worse* driving. The floor tracks total capability and training generation, not active params or bit-width (\$4.5). **(4) Trust floor holds everywhere:** board false-cert is **0% across every model**; the bare gemma-4-12b certifies **11/11 unfixed bugs** (100% false-positive) while the board certifies none — the structural guarantee is starkest exactly where the bare model is most dangerous.

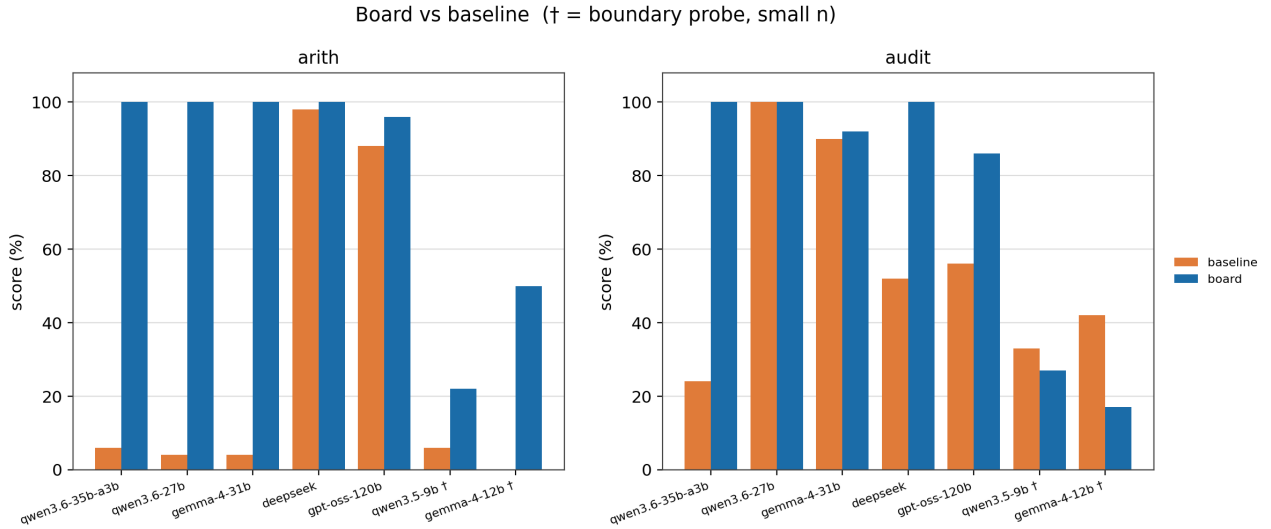


Figure 4: Board vs baseline across models, on arith and audit. The board pulls every clean-tier model to ~100 regardless of its bare score (4-98%); only the boundary probes (†, small n) — where driving collapses — fall short. The lift concentrates exactly where the bare model errs.

6.1 Per-family behaviour

On **arith**, weak/quantized non-thinking models are near-zero bare (baseline 4-10% at the hard setting) and the board takes them to the top of the range — the board supplies the arithmetic the model cannot. On **audit**, even strong cloud models miss errors unaided (a thinking reasoner scored as low as 26% bare) while the board reaches 100% with zero hallucinated findings — the dimension with the largest board win. On **coding-trust** the arms are near-even because the models we ran are honest about fix status; here the board’s value is the *structural guarantee* (a “fixed” must be derivation-backed and is unforgeable), not a score lift.

6.2 Lift tracks the error rate

The wins concentrate where the model errs (audit, arith on weak/quantized non-thinking models) and vanish to a draw where the model is already reliable (coding-trust on honest models; arith on strong arithmetic). This is the lift relation (§4.2) read off the data: **the size of the board win $\approx$ the model's error rate on that dimension**; on a reliable dimension the board contributes an unforgeable guarantee rather than points.

6.3 The driving floor (a core result, not a limitation)

The decisive variable is whether the model can drive. The boundary probes show it cleanly: qwen3.5-9b produces **14/18 empty boards** on arith — board 22% while making *zero* non-empty arithmetic errors — a driving collapse, not a reasoning one. The empty-board failure (§4.4) is the mechanism. A separate probe isolates the cause: pre-ingesting the ground facts (removing the modeling/driving burden, leaving only the board's arithmetic) flips a collapsed run to passing — feeding the board was the problem, never its arithmetic. The collapse is harness-sensitive (§4.5): much of the apparent boundary collapse is format lock-in a fairer harness removes, so we report both honestly — a genuinely collapsed driver yields no lift (and on certify tasks can underperform bare chat), and the boundary failures we *do* report are a disclosed lower bound. We keep “the board does not help / can hurt” as a first-class result, not buried in limitations.

6.4 The sweet spot — and the lowest-compute anchor

qwen3.6-35b-a3b realizes high-error $\times$ clean-driving at the cheapest *per-token* compute we tested: a **3B-active** MoE that takes **arith 6% $\rightarrow$ 100% and audit 24% $\rightarrow$ 100%** under the board, with 0 empty boards / 0 DNF across all three families (n=50). A model contributing essentially none of the correct arithmetic becomes exact once connected to the board — and it does so at **3B active params, fewer than the dense 9-12B models that collapse** (§4.5). qwen3.6-27b (4-bit) is the same story at the dense tier (arith 4% $\rightarrow$ 100%). The sweet spot is therefore not one lucky model but a *region*: current-generation small models, quantized and local, drive the board cleanly and inherit its exactness — the value lands where deployment is cheapest.

6.5 Quantization (resolved)

The quantization gradient is now in, and it is a **null result: within an official quantization, bit-width does not move driving**. qwen3.6-27b reaches arith/audit 100 at 4-bit, fp16, and fp8 — identical board score, zero empty boards at every precision. gemma-4-31b is likewise clean at 4-bit and fp16 (audit 92/98). Even two 4-bit K-quant mixes of the anchor (qwen3.6-35b-a3b Q4_K_S vs Q4_K_M) differ by a single audit case (49 vs 50 of 50) — noise, not a precision effect. The board's output is quantization-independent by construction (the kernel computes exactly regardless of model precision), and the *driving* side now shows no bit-width sensitivity either. An earlier draft reported a “crude-4-bit collapses driving” contrast; that was a harness artifact (§4.5) and is **withdrawn**. The floor that remains is the training-generation one (§4.5): older/smaller models trip, current-generation ones drive, at any official precision.

6.6 Cost — and the redundancy of thinking

The board arm is input-heavy (the board view is re-sent across turns) and slower in wall-clock; tokens, not wall-clock, are the cross-hardware comparable unit (serving speed confounds wall-clock). But the cost story has a twist that is itself a result. **Hold the board fixed and toggle the model's thinking: the board score does not move**. Board + non-thinking = board + thinking = **100** on every clean-tier variant we ran (qwen3.6-27b, deepseek-v4-flash, gemma-4-31b; n=50). Thinking adds nothing on top of the board, because the board already supplies the exact, mechanical half of reasoning a thinking chain would otherwise grind out.

Now toggle thinking on the **bare** model. It lifts the bare score but **does not reach the board’s ceiling** on structural tasks — bare + thinking still scores audit **26%** and hard-arithmetic **86%** (deepseek-v4-flash, n=50; as low as 58% on qwen3.6-27b) against the board’s 100 — while spending **several-fold more tokens**: a thinking baseline emits **5-22×** the output tokens of its non-thinking sibling (285K vs 51K on deepseek arith; 440K vs 20K on gemma-4-31b arith). The two levers are therefore **not substitutes**: thinking buys the bare model more tokens and a higher-but-still-short score; the board buys the *ceiling* at non-thinking cost. For the structural class the board covers, **thinking is redundant** — you pay a multiple for reasoning the board already does exactly. (Scoped: thinking still helps *capacity* tasks — open-ended judgement — which the board does not touch, §4.1.)

What the board’s tokens buy, then, is exactness, non-fabricability, and — for a model otherwise 0/10 — an answer at all, reached at the cheapest non-thinking, low-active, quantized-local tier.

6.7 Belief revision

The board is provably exact and scale-invariant; the model is competent but degrades and is unauditale. We run the revision family on deepseek-v4-flash — our cleanest-driving model — across four problem scales (n = 50/scale).

avg. conclusions / case	retractions	board	baseline (per-case)	baseline (per-conclusion)	model over-retractions
8	1-2	50/50 (100%)	45/50 (90%)	96.5%	9
13	2-3	50/50 (100%)	43/50 (86%)	94.7%	24
19	3-4	50/50 (100%)	37/50 (74%) *	93.9%	39
27	4-6	50/50 (100%)	43/50 (86%)	94.0%	55

(board 0 errors at every scale; * = Wilson 95% CIs disjoint. A same-difficulty replicate at 19 conclusions scored baseline 80% vs the 74% above — indicating $\approx \pm 6$ pp per-case noise at n = 50; the per-conclusion column and the error count are the stabler signals.)

The **board is 100% at every scale, with zero errors**: the model drives it cleanly (4-6 turns, no empty boards — the *driving floor* generalizes to this domain), and the truth-maintenance layer withdraws exactly the unsupported conclusions while retaining every redundant- and independent-support one.

The model’s **baseline (in-context revision) is not a failure but a degrading approximation**. Per queried conclusion it is 96.5% at the smallest scale and falls to ~94% as the belief network grows; equivalently its per-conclusion error roughly doubles (3.5% → 6%) and its absolute over-retraction count grows ~six-fold (9 → 55). The all-or-nothing per-case rate is correspondingly noisy (74-90%) but never reaches the board’s exactness. The failure is **one-directional over-retraction**: the model **withdraws access it should have kept** (dropped-live errors outnumber kept-dead by ~26:1), because it fails to notice that a revoked role’s actions are still enabled by a surviving role.

This is a **trust** result, not a score lift. A strong model revises beliefs well enough to *look* reliable (~95%), but it is (i) never *provably* correct, (ii) *unauditale* — there is no derivation to inspect — and (iii) *degrading with scale*, exactly where the board’s exact, evidence-linked revision is scale-invariant. In settings where a wrong revision carries cost — access control, compliance, safety interlocks — a 95%-and-falling black box is not enough; an auditable, provably exact revision system is a different trust object. The board’s value here is the **structural guarantee** (Contribution 2), sharpened by the fact that the model’s reliability *erodes as the problem grows*.

7. Limitations & Threats to Validity

We make the main threats explicit. **(a) Pilot scale**: n=50/family, single machine, narrow synthetic tasks, single run per cell. **(b) Quantization confound in “0→”**: part of a low *bare* score

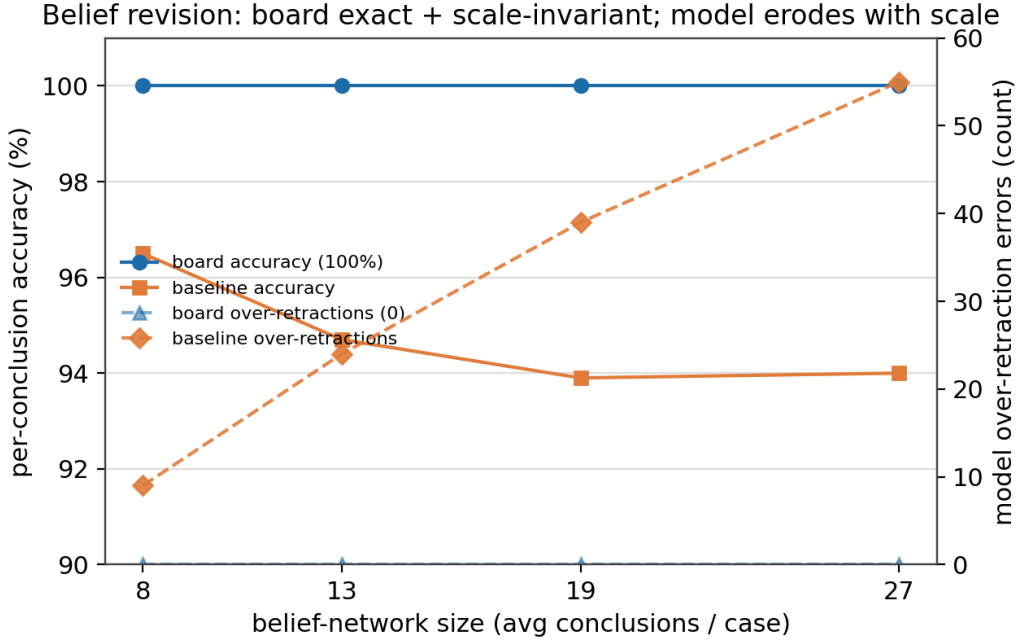


Figure 5: Belief revision vs problem size on deepseek-v4-flash ($n = 50/\text{scale}$). The board is exact and scale-invariant — flat 100% accuracy, 0 over-retractions; the model is competent but *erodes* with scale, per-conclusion accuracy slipping 96.5% $\rightarrow$ 94% while over-retraction errors grow 9 $\rightarrow$ 55.

on a quantized model may be quantization rather than pure inability; the board-side 100% is unaffected, and the fp16 non-thinking baseline (full precision, also low) is the cleaner control, now re-run (§6.5). **(c) The driving floor is model/training-relative**, not an absolute threshold and not a function of per-token compute (a 3B-active current-gen model drives cleanly while larger-active older ones collapse). **(d) The scorer is fragile:** our own baseline negation judgement once misfired; caught, fixed, regression-guarded — auditing our own measurement is a validity point, not a hidden one. **(e) coding-trust is a draw** on honest models; the value there is the structural guarantee, not score.

(f) Data maturity: the fp16/fp8 gradient is now in and resolves to a **null result** (§5.1, §6.5); the boundary probes (gemma-4-12b, qwen3.5-9b) are small-n, strict-harness disclosed lower bounds (§5.5). **(g) “Airtight” is scoped to evidence-provenance adjudication, not a minimum trust floor:** the board never passes a fabricated or mislabeled conclusion, but auto-done (receipt.closed) reports the grounding tier rather than blocking on it, so the self-driven completion path bypasses two layers the manual done path runs (a trusted required_floor gate and the asserted-finding critique) — a known, bench-pending asymmetry that corrupts no completion claim. **(h) Functional-dependency conflict detection is opt-in:** the arith bench deliberately drops the functional_dependency seed (it hurt weak-model driving), so p10-style bare-assertion pollution is reachable by default unless a dependency is declared. A full campaign must add: more $n \times$ seeds with statistics, more models and quantization tiers, broader and more standard tasks, stronger baselines (CoT / tool-use / code-interpreter, not just bare chat), and ablations isolating each board component (derivation gate / grounding floor / FD conflicts).

8. Conclusion

The board replaces the mechanical, deductive half of reasoning — exact arithmetic, consistency, refusing the underived — while leaving judgement in the weights. The binding constraint is not the model’s reasoning but its **driving**: whether it can feed the symbolic interface, a capability

separable from reasoning and, in our pilot, tracking training generation rather than bit-width or serving mode. The practical upshot is that a good cheap 4-bit local model plus the board can recover exact, non-fabricable reasoning at the cheapest deployment tier. The mental model is one line: **the model proposes (models + drives); the board reasons (derives, checks consistency, computes exactly, refuses the underived)**. We release the system and reproducible probes at <https://github.com/rulith-dev/rulith>.

Appendix: full results matrix

Table A1 — full results matrix. Cells = board% / baseline%; * = disjoint Wilson 95% CIs (board vs baseline). n=50/family except qwen3.6-27b (fp16, think) (n=36) and the small-n boundary probes. The **main Table 1 is the headline subset**; this table adds the full precision × thinking gradient and the partial / boundary drivers. Precision convention as in Table 1 (unlabeled = 4-bit non-thinking; cloud = full-precision). Three excluded runs are noted in §5.1.

model	arith b/base	audit b/base	coding cert	empty-bd (ar/au)	false-cert
deepseek-v4-flash (nt)	100/98	100/52 *	100/100	0 / 0	0%
deepseek-v4-flash (think)	100/86	100/26 *	100/100	0 / 0	0%
qwen3.6-35b-a3b (3B-active MoE)	100/6 *	100/24 *	100/100	0 / 0	0%
qwen3.6-27b	100/4 *	100/100	100/100	0 / 0	0%
qwen3.6-27b (fp16, nt)	100/6 *	100/100	100/100	0 / 0	0%
qwen3.6-27b (fp16, think)	100/58 *	100/97	100/100	0 / 0	0%
qwen3.6-27b (fp8, nt)	100/10 *	100/98	100/100	0 / 0	0%
qwen3.6-27b (fp8, think)	100/96	100/100	100/100	0 / 0	0%
gemma-4-31b	100/4 *	92/90	100/100	0 / 0	0%
gemma-4-31b (fp16, nt)	100/4 *	98/94	100/100	0 / 0	0%
gemma-4-31b (fp16, think)	100/96	100/100	100/100	0 / 0	0%
gpt-oss-120b (lowthink)	96/88	86/56 *	100/100	0 / 0	0%
<i>partial / boundary drivers:</i>					
gemma-4-26b-a4b (4B-active MoE)	78/32 *	70/84	96/100	3 / 0	0%
gemma-4-12b (12B dense)	50/0 *	17/42	100/100	6 / 2	0%
qwen3.5-9b (9B dense)	22/6	27/33	100/100	14 / 5	0%

Reading it: every clean-tier model — cloud, 4-bit, fp16, fp8, thinking or not — drives to **board 100 on arith and audit**, so the precision × thinking gradient is flat (\$6.5–6.6). The partial / boundary drivers are where driving degrades: board ties or dips and empty boards appear. They track **training generation, not active params** — the 4B-active gemma-4-26b-a4b drives worse (78 / 70) than the 3B-active anchor (\$4.5). Board false-cert is **0% on every row**.

Acknowledgments

This manuscript — its drafting, figures, and data tabulation — was prepared with the assistance of an AI assistant (Claude). The system (rulith), the experimental design, the runs, and the results are the author’s own, and the author is responsible for all claims.

References

(All arXiv IDs, titles, and authors verified against arXiv.) - **[G-SPEC]** Divya Vijay, Vignesh Ethiraj. Graph-Symbolic Policy Enforcement and Control (G-SPEC): A Neuro-Symbolic Framework for Safe Agentic AI in 5G Autonomous Networks. arXiv:2512.20275. - **[CompiledAI]** Geert Trooskens et al. Compiled AI: Deterministic Code Generation for LLM-Based Workflow Automation. arXiv:2604.05150. - **[PlanCompiler]** Pranav Harikumar. PlanCompiler: A Deterministic Compilation Architecture for Structured Multi-Step LLM Pipelines. arXiv:2604.13092. - **[Blueprint]** Libin Qiu et al. Blueprint First, Model Second: A Framework for Deterministic LLM Workflow. arXiv:2508.02721. - **[CEGIS]** Sumit Kumar Jha et al. Neuro-Symbolic Reasoning for Planning: Counterexample-Guided Inductive Synthesis using Large Language Models and Satisfiability Solving. arXiv:2309.16436. - **[PaperTrail]** Anna Martin-Boyle, Cara A. C. Leckey, Martha C. Brown, Harmanpreet Kaur. PaperTrail: A Claim-Evidence Interface for Grounding Provenance in LLM-based Scholarly Q&A. arXiv:2602.21045 (CHI’26). - **[AgentTraces]** Yiqi

Wang et al. From Agent Traces to Trust: A Survey of Evidence Tracing and Execution Provenance in LLM Agents. arXiv:2606.04990.

Appendix — Reproducibility

Open system + probes at <https://github.com/rulith-dev/rulith> (npm: `rulith`); the bench matrix and per-arm scoring contracts are in `docs/benchmarks.md`, and the run-to-model mapping is in `docs/papers/data-manifest.md`. The sweet-spot run reproduces with (PowerShell, local 27B non-thinking via LM Studio):

```
$env:RULITH_LLM_MODEL="qwen/qwen3.6-27b"; $env:RULITH_LLM_BASE_URL="http://127.0.0.1:1234/v1"
$env:RULITH_LLM_TIMEOUT_MS="600000"; $env:RULITH_MAX_TOKENS="32000"; $env:RULITH_BENCH_N="50"
$env:RULITH_BENCH_UNIT_DIGITS="8"; $env:RULITH_BENCH_QTY_DIGITS="5"
npm run verify:bench-arith
npm run verify:bench-audit
npm run verify:bench-coding-trust
```

The aggregate matrix (Wilson CIs, board vs baseline, token tallies) is produced by `npx tsx src/examples/bench-aggregate.ts` over the per-arm run logs.